

Capitolo 3

Stato dell'arte e research gap

Obiettivo del capitolo

Il Capitolo 2 ha mostrato che la threat analysis può iniziare prima dell'implementazione, che le metodologie osservano il sistema attraverso rappresentazioni differenti e che traceability, provenance, revisione e cambiamento condizionano la validità dei risultati. Questo capitolo sposta l'attenzione dalla definizione dei concetti al confronto fra gli approcci esistenti. L'obiettivo non è stabilire quale metodologia sia migliore in assoluto, ma verificare quali parti della pipeline necessaria a un processo documentation-driven siano già coperte dalla letteratura e quali rimangano invece aperte.

Il confronto è costruito attorno alla domanda centrale della tesi: se un progetto possiede documentazione governata, è possibile derivarne una rappresentazione analizzabile e metodologicamente neutrale, permettere a metodi differenti di operare sulla stessa conoscenza di sistema e riportare i risultati accettati nella documentazione come requisiti di sicurezza tracciabili? La risposta non può essere ricavata da un singolo filone. Gli approcci requirements-first mostrano come partire da requisiti e scenari; gli approcci goal-oriented e problem-frame formalizzano intenzioni, contesto e assunzioni; il model-driven security sposta l'automazione su modelli espliciti; l'automated threat modelling applica cataloghi e regole dopo che un modello tipizzato è disponibile; l'automated requirements engineering affronta invece la trasformazione a monte da testo a artefatti candidati; la traceability literature studia come preservare relazioni e origine; gli studi più recenti sugli LLM mostrano infine che leggibilità, correttezza semantica, coverage, groundedness e riproducibilità sono proprietà distinte.

Il capitolo mantiene una distinzione epistemica importante. Quando una fonte mostra che una tecnica è applicabile prima del codice, questo non viene trasformato nella conclusione che la documentazione grezza sia sufficiente. Quando un tool produce automaticamente threat candidate, questo non viene trasformato nella conclusione che

il processo sia autonomo o completo. Quando una trasformazione LLM produce un output plausibile, questo non viene assimilato a un modello accettato. Il research gap viene quindi derivato dalle discontinuità fra i confini osservati, non da una semplice assenza nominale di un tool chiamato DDTA.

3.1 Dimensioni di confronto

Per rendere comparabili approcci nati con obiettivi diversi, questa tesi utilizza nove dimensioni. Esse non definiscono una tassonomia universale del threat modeling; servono come lente comune per ricostruire dove inizia e dove termina ciascun contributo.

- D1. Starting artifact.** Quale informazione deve esistere prima che il metodo possa essere applicato: testo libero, requisiti, use case, goal model, DFD, grafo tipizzato, inventory di asset, architettura, codice o test model.
- D2. Rappresentazione analitica.** Quale struttura viene trattata come modello operativo del sistema durante l'analisi e quanto essa sia specifica del metodo.
- D3. Focus metodologico.** Quale conoscenza viene privilegiata: scenari ostili, intenzioni, proprietà, asset, componenti e flussi, rischio, requisiti o comportamento eseguibile.
- D4. Semantica method-specific.** Quali categorie, regole, assunzioni o failure mode appartengono al metodo e non alla conoscenza generale del sistema.
- D5. Output.** Quale artefatto viene prodotto: scenario di minaccia, anti-goal, finding, mitigazione, security requirement, configurazione, abstract test o evidenza di esecuzione.
- D6. Boundary di automazione.** Quale trasformazione è realmente automatizzata e quale conoscenza deve essere già disponibile prima dell'automazione.
- D7. Revisione e acceptance.** Dove rimane necessaria una decisione umana e se l'output automatico è trattato come candidato o come risultato canonico.
- D8. Traceability e provenance.** Se e come sia possibile ricostruire la relazione fra fonte, trasformazione, modello, risultato e artefatti successivi.
- D9. Evoluzione.** Se il metodo considera versioni, regression, stale state o propagazione dell'impatto quando cambiano documentazione, modello o ambiente di esecuzione.

Queste dimensioni permettono di evitare un confronto riduttivo basato sul solo nome delle metodologie. Due approcci possono entrambi “identificare minacce” ma richiedere

input incompatibili, produrre output a granularità differente o collocare la revisione umana in punti diversi. Analogamente, due tool possono generare lo stesso numero di candidate ma associarli a unità di modello differenti, rendendo il conteggio non confrontabile come misura di qualità (Granata and Rak, 2024).

3.2 Approcci requirements-first e security requirements

Una prima famiglia rilevante per DDTA parte da artefatti che appartengono direttamente alla fase dei requisiti. Il vantaggio di questi approcci è evidente rispetto all'obiettivo di anticipare la sicurezza: non richiedono che il sistema sia già implementato. Il limite, per il problema specifico di questa tesi, è che normalmente assumono che il materiale iniziale sia già strutturato nella forma richiesta dal metodo.

3.2.1 Misuse case e security use case

Sindre and Opdahl (2005) estendono gli use case funzionali introducendo misuser, misuse case e security use case. Il percorso è intuitivo: il comportamento desiderato viene affiancato da scenari ostili, i misuse case minacciano use case legittimi e i security use case mitigano i comportamenti dannosi. Il metodo è significativo perché collega direttamente il comportamento funzionale alla motivazione di sicurezza e può essere applicato durante requirements elicitation.

La tecnica copre quindi una parte importante della pipeline cercata da DDTA: *functional behavior* $\rightarrow$ *threat scenario* $\rightarrow$ *security response*. Tuttavia, il metodo non definisce una trasformazione da documentazione eterogenea a modello di scenario. Gli analisti devono identificare attori, misuser, asset, obiettivi, precondizioni, percorsi dannosi e mitigazioni. Le linee guida di discovery rimangono aperte e il metodo non fornisce una condizione globale di completezza. Il risultato è adatto a un overlay scenario-based, ma non costituisce un modello neutrale del sistema utilizzabile automaticamente da metodologie con viewpoint differenti.

3.2.2 Security requirements come vincoli su funzioni

Haley et al. (2008) propongono un framework in cui i security requirements sono vincoli su specifici requisiti funzionali e la loro soddisfazione viene argomentata attraverso un outer argument formale e inner arguments strutturati. L'approccio rende esplicite le premesse di dominio necessarie affinché un requisito sia soddisfatto e utilizza grounds, warrants, rebuttals e trust assumptions per mettere alla prova tali premesse.

Per la tesi, il contributo è duplice. Primo, rafforza la distinzione fra un problema analitico e l'obbligazione di prodotto che ne deriva: un requisito non coincide con la minaccia che lo motiva. Secondo, mostra che un fallimento dell'argomento può diagnosticare informazioni mancanti, requisiti infeasible o necessità di nuove funzioni. Il limite rispetto a DDTA è a monte: il framework richiede contesto, domini, fenomeni condivisi e comportamento, ma lascia aperta la costruzione di tale rappresentazione. Non fornisce quindi il boundary documentazione $\rightarrow$ modello analizzabile.

3.2.3 Problem frame e separazione problema-soluzione

Hatebur et al. (2006) mantengono una separazione ancora più netta fra security problem, concretized requirement, solution principle, mechanism e specification. Il security problem frame richiede domini ambientali, interfacce, fenomeni, assunzioni e capacità dell'attaccante; il passaggio alla soluzione introduce progressivamente principi e meccanismi. Due obblighi di implicazione esprimono l'adeguatezza condizionale fra specifica, requisito concretizzato e requisito originario.

Questa impostazione è rilevante per evitare che un sistema di threat analysis trasformi automaticamente una categoria di minaccia in una prescrizione implementativa. Tuttavia, frame matching, selezione del meccanismo e costruzione del contesto rimangono attività dell'ingegnere. La fonte dimostra il valore di un refinement governato, non una pipeline automatica o multi-metodo.

Nel complesso, gli approcci requirements-first mostrano che il percorso verso i requisiti di sicurezza può iniziare molto presto e può conservare relazioni con il comportamento funzionale. Non mostrano però un common result model attraverso cui risultati provenienti da metodologie eterogenee vengano normalizzati senza perdere la propria semantica. Ogni approccio porta con sé il proprio modello analitico e il proprio percorso verso il requisito.

3.3 Scenario, misuse e anti-goal

Gli approcci scenario-based e goal-oriented condividono la capacità di ragionare su comportamenti o intenzioni prima dell'implementazione, ma utilizzano strutture concettuali differenti. Il confronto è utile perché mostra già, all'interno del solo requirements engineering, che "analizzare la stessa documentazione" non significa necessariamente costruire lo stesso modello.

3.3.1 Anti-model goal-oriented

van Lamsweerde (2004) costruisce un primal model di goal, agenti, oggetti, operazioni, requisiti, aspettative e proprietà di dominio e lo affianca a un anti-model di attaccanti,

anti-goal, capacità e vulnerabilità. L'anti-goal refinement permette di distinguere condizioni realizzabili dall'attaccante da debolezze realizzabili dal sistema attaccato e consente, quando il modello è sufficientemente formalizzato, reasoning e generazione di scenari.

Rispetto ai misuse case, il centro dell'analisi passa dalla sequenza di comportamento all'intenzione e alle condizioni di soddisfazione. Questo fornisce una forma di assurance interna più forte, ma aumenta il costo di modellazione e richiede una traduzione più ricca dalle fonti documentali a un modello formale. Sensitive-object identification, attacker motive, domain properties, risk assessment e selection of countermeasures rimangono responsabilità analitiche. Anche la condizione di arresto globale per la raffinazione di requirement, anti-requirement e contromisure viene lasciata aperta dall'autore.

3.3.2 Executable misuse case

Whittle et al. (2008) aggiungono un'altra prospettiva: use case, EIOD, sequence diagram, attacker modification e mitigation aspects vengono composti fino a generare communicating finite-state machines. Gli attacchi noti possono quindi essere eseguiti a livello di modello e conservati come regression suite. Questo contributo non genera automaticamente nuovi threat; rende invece eseguibile l'evidenza relativa a scenari ostili già modellati.

Il boundary è importante per la tesi. Un finding analitico e una prova eseguibile non sono lo stesso artefatto. Nel lavoro di Whittle et al. gli input ostili provengono da code analysis precedente o brainstorming umano; due attacchi del caso EVS richiedono dettagli di storage o rete non presenti nel modello requirements-level. L'automazione è quindi downstream rispetto alla modellazione, e la copertura è relativa alle tracce rappresentate.

Queste due linee di ricerca evidenziano una tensione che ricompare nel problema DDTA: il livello di struttura che abilita reasoning più forte è anche il livello che richiede più interpretazione prima dell'analisi. Nessuna delle due fonti dimostra che tale struttura possa essere derivata in modo completo e affidabile dalla normale documentazione del progetto.

3.4 Model-driven security e model-based security testing

Il model-driven security sposta il centro del processo su una rappresentazione esplicita del sistema e delle proprietà di sicurezza. La letteratura classica su SecureUML e

model-driven security mostra come modelli UML arricchiti possano diventare input per trasformazioni verso infrastrutture di access control o altri artefatti di sicurezza (Lodderstedt et al., 2002; Basin et al., 2006). Per DDTA, l'aspetto più importante non è la specifica tecnologia di generazione, ma il confine metodologico: l'automazione diventa possibile dopo che un modello semanticamente adeguato è stato costruito.

Questo pattern è confermato dalla Rapid Review di Lonetti et al. (2023) sul model-based security testing per IoT. Nei diciassette studi inclusi, modelli UML/OCL, timed automata, attack tree e altri formalismi vengono arricchiti con proprietà di sicurezza, attack model e criteri di selezione. Da questi modelli si derivano abstract test; per produrre evidenza sul sistema reale servono poi concretizzazione, script, adapter, execution environment e oracle.

La review mostra inoltre che model coverage, attack-class coverage e security completeness sono dimensioni distinte. Più del 40% degli studi utilizza criteri di coverage del modello, circa il 30% combina verification e testing e oltre il 30% integra MBST con tecniche quali penetration testing, fuzzing o model learning. Nonostante ciò, gli studi selezionati coprono soltanto una parte della tassonomia di attacchi adottata e rimangono prevalentemente esplorativi (Lonetti et al., 2023).

Per il research gap, il model-driven security copre bene il tratto *structured model* → *security artifact/evidence*. Non copre invece il passaggio documentazione → modello neutrale, né risolve la coesistenza di più metodologie sulla stessa base informativa. La sua lezione è piuttosto che ogni trasformazione automatica deve dichiarare il contratto dell'input e che l'evidenza derivata rimane relativa a quel contratto.

3.5 Automated threat modeling

L'automated threat modeling è il filone più vicino, nominalmente, alla funzione che verrà implementata in *ThreatForge*. Tuttavia, la review di Granata and Rak (2024) mostra che l'automazione studiata opera soprattutto *dopo* che il sistema è stato rappresentato come modello tipizzato.

La review analizza cinquantacinque studi e classifica model types, threat classifications, selection mechanisms e tool. DFD e grafi sono rappresentazioni dominanti; STRIDE è la classificazione più frequente; label e relationship-based catalogue selection sono meccanismi comuni. Alcuni approcci utilizzano inoltre proprietà, protocolli, ruoli delle relazioni, propagation, rule matching e pattern più ricchi. Il punto comune è che una regola può selezionare minacce soltanto perché gli elementi analizzati hanno già tipo e proprietà sufficienti.

Il confronto WordPress fra Microsoft tooling, Threat Dragon, SLA-generator e PyTM è particolarmente istruttivo. Gli strumenti associano threat a unità differenti del modello

e generano liste con scope e granularità diverse. Un raw candidate count non può quindi essere interpretato come misura di completezza o qualità. La stessa categoria può essere istanziata a livello di componente, flow o relazione e produrre numeri non direttamente confrontabili (Granata and Rak, 2024).

Il boundary scientifico può essere espresso così:

$$\textit{typed system model} + \textit{catalogue/rules} \rightarrow \textit{candidate threats}.$$

La review non stabilisce invece:

- come una documentazione eterogenea diventi il typed model richiesto;
- come dimostrare che il modello contenga tutta l'informazione necessaria alla metodologia;
- come confrontare semanticamente output generati da tool con granularità differenti;
- come trasformare finding accettati in requisiti di sicurezza indipendenti dalla tassonomia del tool;
- come propagare in modo esplicito lo stale state quando cambia una fonte documentale da cui il modello era stato derivato.

L'automated threat modelling costituisce quindi un elemento essenziale del futuro overlay, ma non risolve il problema documentation-driven a monte.

3.5.1 STRIDE-AI come controprova asset-centered

Il paper di Mauri and Damiani (2022), letto integralmente come riferimento metodologico mirato, mostra in modo concreto perché un modello comune non può essere ridotto a un DFD o a un semplice inventory di componenti. STRIDE-AI è definito dagli autori come metodologia asset-centered per sistemi AI/ML. Il ciclo di vita considerato parte dai requirements e attraversa Data Management, Model Learning, Model Tuning, Model Deployment e Model Maintenance; gli asset vengono organizzati nelle macro-categorie Data, Models, Actors, Processes, Tools e Artefacts.

La metodologia usa FMEA per associare a ciascun asset funzioni, prerequisiti, failure mode, cause ed effetti e mappa poi le failure mode a proprietà ML-specifiche e categorie STRIDE. Il caso TOREADOR mostra che, per un training data stream, dettagli come autenticazione fra piattaforme, firme prodotte dai data logger e momento in cui vengono aggiunte le label modificano direttamente i rami plausibili delle threat tree. Il finding dipende quindi da conoscenza funzionale, architetturale e operativa specifica, non dalla sola etichetta dell'asset (Mauri and Damiani, 2022).

STRIDE-AI non viene assunto come fondamento universale di DDTA. Al contrario,

è utile come controprova: se il core comune venisse modellato soltanto secondo le entità tipiche di un approccio software/component-centered, potrebbe non esporre le informazioni su training data, hyper-parameters, model artifacts e funzioni richieste dal metodo. Se invece incorporasse direttamente tutte le categorie e i mapping STRIDE-AI, perderebbe neutralità metodologica. La tesi colloca il proprio problema precisamente fra questi due estremi.

3.6 Threat-model-as-code e tool support

L'espressione *threat-model-as-code* viene usata in questa tesi in senso operativo: la rappresentazione analitica è conservata come artefatto strutturato, versionabile e processabile automaticamente, invece di vivere soltanto in un documento statico o in un diagramma non interrogabile. Questa caratteristica può rendere più semplice l'integrazione con version control, CI e controlli deterministici, ma non risolve da sola il problema della qualità del modello.

La comparazione riportata da Granata and Rak (2024) include approcci in cui il modello è descritto attraverso DSL, codice o rappresentazioni strutturate e poi processato da regole o cataloghi. PyTM è un esempio del fatto che la costruzione del threat model può essere resa programmabile; altri tool utilizzano modelli grafici o formati propri. La differenza di encoding non elimina il requisito principale: l'autore del modello deve sapere quali componenti, trust boundary, data flow, proprietà e relazioni inserire.

Questa osservazione limita una possibile interpretazione del gap. DDTA non nasce perché i modelli esistenti non siano versionabili. Il problema è che un modello threat-specific scritto in codice rimane un artefatto ulteriore che può divergere dalla documentazione del progetto e può incorporare già la semantica della metodologia scelta. Versionare il modello migliora la gestione dell'artefatto; non stabilisce che esso sia derivato in modo tracciabile dalla fonte autorevole né che sia riusabile da overlay con presupposti differenti.

Un approccio documentation-driven richiede quindi una relazione più forte di “il threat model è nel repository”. Deve poter distinguere almeno: documentazione governata; evidenza estratta; elementi candidati; modello canonico revisionato; configurazione del metodo; risultati method-specific; finding normalizzati; requisiti di sicurezza derivati. È questa catena, non il formato testuale o grafico del singolo threat model, che il research gap deve rendere esplicita.

3.7 Automated requirements engineering e modelli candidati derivati da documenti

Gli studi sull'automated requirements engineering affrontano il lato opposto della pipeline rispetto all'automated threat modelling: invece di applicare regole a un modello già tipizzato, cercano di trasformare testo e artefatti requirements-level in strutture più formali.

La systematic review di Umar and Lano (2024) classifica ottantacinque studi empirici su strumenti di supporto automatizzato alla Requirements Engineering. Il 94% usa testo in linguaggio naturale come input. UML è l'output modellistico più frequente, ma gli strumenti svolgono anche omission detection, consistency checking, classification, duplicate detection, quality analysis, mining e riuso. Il pattern rilevante è quindi più ampio di text-to-UML: documenti non strutturati o semi-strutturati possono alimentare trasformazioni verso artefatti candidati utili al processo software.

La review mostra però che la maggioranza degli strumenti è semi-automatica e one-shot. Anche le trasformazioni descritte come fully automated producono output destinati a verifica umana nel processo più ampio. Gli autori identificano come problemi aperti la traceability verso le source statements, l'evoluzione dei modelli, la human-in-the-loop governance, la validazione industriale e la disponibilità di benchmark comuni (Umar and Lano, 2024). Questo insieme di limiti coincide con il tratto che DDTA deve affrontare a monte della Base Analysis.

3.7.1 Requirements-to-architecture

Il mapping study di Souza et al. (2019) esamina trentanove studi sulla derivazione di modelli architetturali da specifiche dei requisiti. I textual requirements costituiscono il gruppo di input più numeroso, ma compaiono anche goal model, feature model, problem frame e sequence diagram. Le trasformazioni sono representation-specific e nessun approccio completamente automatizzato viene identificato.

Le decisioni su alternative architetturali, quality attributes e trade-off rimangono fortemente dipendenti dalla conoscenza tacita dell'architetto. Le viste e le ADL impiegate sono eterogenee e la traceability esplicita requirements-to-architecture è rara nel corpus mappato. Il risultato è importante perché impedisce una scorciatoia concettuale: generare una struttura coerente a partire dai requisiti non significa aver derivato l'architettura corretta o aver dimostrato la soddisfazione dei requisiti (Souza et al., 2019).

Per DDTA la conseguenza è che una eventuale Base Analysis non deve diventare un contenitore di decisioni architetturali implicite. Quando una relazione non è

direttamente espressa dalle fonti, il sistema deve poter distinguere estrazione, inferenza e decisione revisionata.

3.8 LLM-derived candidate models e requisiti

Gli studi più recenti sugli LLM permettono di osservare direttamente la trasformazione da documenti o requisiti a artefatti strutturati. Essi sono particolarmente rilevanti perché mostrano errori che un sistema documentation-driven deve rappresentare esplicitamente invece di nasconderli dietro un output sintatticamente valido.

3.8.1 Sequence diagram da requisiti

Ferrari et al. (2024) studiano GPT-3.5 su ventotto documenti requirements realistici appartenenti a diciotto domini e costruiscono ottantasette varianti contenenti additions, removals, modifications, ambiguity, inconsistency e incompleteness. Gli output PlantUML risultano generalmente forti per standard adherence, understandability e terminology, ma la correctness non supera significativamente il valore neutro e la completeness rimane imperfetta.

La failure taxonomy derivata dagli autori mostra omissioni di condizioni, contraddizioni nascoste, gestione errata di numeri e tempi, comportamento assegnato al componente sbagliato, termini inventati e trace annotations incomplete. L'importanza per DDTA è metodologica: un diagramma leggibile e renderizzabile può essere un *candidate artifact* utile, ma non un modello canonico finché non sono state controllate source coverage e fedeltà semantica.

3.8.2 Class diagram e separazione syntax-semantics-coverage

Garaccione et al. (2025) confrontano GPT-4o, DeepSeek v3, Gemini 2.5 Pro e Qwen 3 su quindici esercizi universitari per la generazione di class diagram. Lo studio separa syntax errors, semantic errors e completeness rispetto a soluzioni di riferimento. I risultati non identificano un vincitore universale: modelli diversi hanno profili differenti e molte differenze pairwise non sono statisticamente significative.

Il dato più rilevante per il processo è qualitativo. Association type e multiplicity dominano gli errori semantici; un output parser-compatible può quindi contenere containment, generalization, direction o cardinality errate. La validazione sintattica è necessaria ma non sufficiente. Una pipeline governata deve poter conservare separatamente il fatto che l'output sia parseable, che rappresenti gli elementi attesi e che le relazioni corrispondano alla fonte.

3.8.3 Requirement extraction, correttezza locale e coverage

Abualhaija et al. (2025) offrono un caso particolarmente chiaro della differenza fra qualità locale e copertura globale. Sei esperti costruiscono reference set di 61 requisiti di accesso e 47 di portabilità a partire da GDPR e fonti complementari; XTRAREG genera requirement candidate, riferimenti e rationale usando GPT-3.5 o GPT-4o in configurazioni zero-shot o RAG.

Con GPT-4o, una quota elevata degli output prodotti è corretta o parzialmente corretta, ma il numero di reference requirements effettivamente coperti rimane molto più basso. Le citazioni possono inoltre essere incomplete o non grounded. RAG migliora diversi indicatori ma non produce un corpus completo oltre gli articoli direttamente rilevanti. Il dominio regolatorio resta fuori dall’implementazione della tesi, ma il risultato è trasferibile come lezione di processo: **candidate acceptance rate, source coverage e grounding non sono la stessa metrica.**

La combinazione dei tre studi porta a un boundary più preciso rispetto alla generica etichetta “LLM-assisted modeling”:

document → *generated candidate* → *syntax check* → *coverage/grounding check* →
semantic review → *accept/reject/correct*.

La letteratura fornisce evidenza per i primi passaggi e per i failure mode, ma non stabilisce che questa pipeline produca automaticamente una Base Analysis neutrale, completa e continuamente sincronizzata.

3.9 Maintenance, provenance e riproducibilità

Il problema DDTA non termina con la prima analisi. Se documentazione, modello e metodologia evolvono, occorre sapere quali risultati dipendano dalle informazioni modificate. La traceability literature e gli studi sulla riproducibilità mostrano che questo requisito è distinto dalla semplice persistenza degli artefatti.

3.9.1 Traceability prima e dopo la specifica

La definizione classica del requirements traceability problem di Gotel and Finkelstein (1994) mette al centro la capacità di descrivere e seguire la vita di un requisito in entrambe le direzioni. La review di Mucha et al. (2024) mostra che, nella pre-requirements-specification traceability, provenance, responsabilità, impact analysis, versioning, trust e adaptability rimangono temi centrali. La traceability non è quindi un problema risolto dalla sola esistenza di hyperlink.

Lo studio empirico di Ruiz et al. (2023) rileva che, nella popolazione analizzata, la traceability viene ancora percepita come costosa e spesso gestita manualmente. Il valore dipende dall'integrazione nei workflow reali e dalla possibilità di automatizzare attività ripetitive senza togliere il controllo sulle decisioni. Per DDTA, ciò suggerisce che i link devono essere prodotti come parte naturale della trasformazione, non aggiunti ex post come documentazione accessoria.

Großer et al. (2022) mostrano inoltre che le relazioni possono avere semantica a livello di documento, requirement set e singolo requisito. Una normalizzazione in semplici edge non tipizzati può perdere informazione. Questo è direttamente rilevante quando una Base Analysis deriva da documenti: la provenance deve poter indicare non soltanto “da quale file” provenga un elemento, ma quale porzione o asserzione abbia giustificato la derivazione e in quale versione.

3.9.2 Automated trace-link recovery

La possibilità di generare automaticamente i link non elimina il problema della loro validità. Wang et al. (2024) mostrano che il ranking dei metodi per requirement-to-code traceability link recovery cambia con dataset, threshold, classifier e training strategy e che l’F1 può rimanere modesto sui dataset più grandi. Hey et al. (2024) mostrano che preprocessing e classificazione a monte modificano direttamente quali elementi raggiungono il recovery stage, introducendo trade-off fra precision e recall.

Per una pipeline DDTA, un link suggerito automaticamente deve quindi avere stato e provenance propri. Confondere candidate trace e accepted trace produrrebbe un errore strutturale: la stessa relazione usata per propagare stale state potrebbe essere incerta o falsa.

3.9.3 Riproducibilità degli LLM commerciali

Angermeir et al. (2026) introducono un problema temporale ancora più forte. Gli autori ispezionano ottantacinque studi ICSE 2024 e ASE 2024 selezionati secondo il loro protocollo; sessantanove usano servizi OpenAI. Diciotto artifact package sembrano inizialmente adatti alla riproduzione, ma soltanto cinque risultano sufficientemente completi ed eseguibili. Nessuno dei cinque viene riprodotto completamente: due casi sono parziali e tre divergono.

I blocker principali comprendono artifact incompleti, versioni delle dipendenze, difetti del codice e documentazione insufficiente; la deprecazione dei modelli commerciali aggiunge un failure mode specifico. Gli autori osservano inoltre variazioni rilevanti fra run ripetuti. La conseguenza per DDTA non è che ogni analisi debba essere ripetuta trenta volte, né che una tesi debba costruire un framework universale di

scientific reproducibility. È più limitata e concreta: quando una trasformazione dipende da un LLM, devono rimanere distinguibili input, prompt, model/interface identity, dependencies, raw output, intervention history, evaluator e rerun result.

Questo risultato completa la nozione di stale state. Un risultato può diventare stale non soltanto perché è cambiata la documentazione del progetto, ma perché è cambiata la trasformazione che lo ha prodotto o non è più ripetibile nello stesso ambiente. La tesi valuterà il primo tipo di change impact come requisito centrale; il secondo rimane un requisito di provenance e future compatibility per le operazioni LLM-assisted effettivamente usate.

3.10 Confronto trasversale delle fonti

La Tabella 3.1 sintetizza i boundary osservati nelle fonti più direttamente rilevanti. La colonna finale non è una critica agli studi: identifica semplicemente quale tratto della pipeline DDTA rimane fuori dal rispettivo scope.

Tabella 3.1: Confronto source-specific rispetto alla pipeline documentation-driven.

Fonte	Input richiesto	Boundary coperto	Output / evidenza	Boundary non coperto
Sindre–Opdahl (2005)	Use case, attori, asset, scenari	Functional scenario → misuse/security use case	Threat scenario e mitigazione collegati alla funzione	Derivazione da documentazione eterogenea; multi-method common core; completezza globale
van Lamsweerde (2004)	Goal, agenti, oggetti, proprietà di dominio	Primal model → anti-model e refinement	Anti-goal, anti-requirement, vulnerabilità, scenari	Costruzione automatica del goal model; universalità; stopping globale
Haley et al. (2008)	Contesto, domini, fenomeni, comportamento	Security requirement → satisfaction argument	Premesse, rebuttal, trust assumptions, diagnosi	Document-to-context; normalizzazione multi-metodo
Whittle et al. (2008)	Scenari precisi, EIOD, sequence/state information	Known attack model → executable regression	Trace ostili eseguibili a livello di modello	Threat discovery; low-level details mancanti; prova globale di sicurezza
Hatebur et al. (2006)	Problem frame, domini, interfacce, requirement	Security problem → resolution principle → specification	Refinement condizionale verso meccanismo	Derivazione automatica del frame; selezione automatica del meccanismo
Granata–Rak (2024)	DFD/grafo/tipo model	Typed model + catalogue/rules → candidate threats	Liste e mapping di threat	Raw documentation → typed model; comparabilità semantica; accepted findings
Mauri–Damiani (2022)	Lifecycle ML, asset, funzioni, architettura, failure mode	Asset, FMEA, proprietà → threat STRIDE-AI	Threat e threat tree nel caso TOREA-DOR	Modello neutrale condito; valutazione comparativa controllata

Fonte	Input richiesto	Boundary coperto	Output / evidenza	Boundary non coperto
Lonetti et al. (2023)	Security-enriched test model	Model → abstract security test → execution chain	Test implementation-level evidence	Document-to-model; complete attack coverage; generic multi-method analysis
Umar-Lano (2024)	Principalmente requisiti in linguaggio naturale	Document → candidate RE artifact	UML, classification, quality finding, structure RE	Neutralità, traceability completa, evoluzione e sincronizzazione
Souza et al. (2019)	Textual/modelled requirements	Requirement representation → candidate architecture	Architectural view o model candidate	Decision rationale automatizzato; trade-off; strong traceability; unique correct architecture
Ferrari et al. (2024)	Requirements document	Text → LLM sequence-diagram candidate	PlantUML leggibile ma fallibile	Gold model; semantic acceptance; continuous synchronization
Garaccione et al. (2025)	Testo strutturato di esercizi	Testo → candidate class diagram multi-LLM	Misure di syntax, semantics e completeness	Generalizzazione a progetti reali; provenance; run ripetuti; security analysis
Abualhaija et al. (2025)	Norme e reference corpus	Legal text → requirement candidate	Correctness, coverage, groundedness evidence	Complete corpus extraction; automatic legal acceptance; thesis legal scope
Mucha / Ruiz / Großer	Artefatti e relazioni di progetto	Source/artifact relation → traceability	Provenance, responsibility, relation semantics	End-to-end DDTA dependency semantics; automatic accepted traces
Angermeir et al. (2026)	Studio pubblicato e artifact package	Artifact → valutazione di rerun indipendente	Stati reproduced, partial o divergent; blocker evidence	Replay stabile universale; garanzie deterministiche

Il confronto mette in evidenza tre pattern.

Il primo è una **frammentazione per boundary**. Le fonti coprono in modo convincente tratti specifici: scenario-to-threat, goal-to-anti-goal, model-to-threat, model-to-test, document-to-candidate-model, trace-link recovery o artifact-to-rerun. Nessuna fonte del corpus copre l'intera catena dalla documentazione governata fino al requisito di sicurezza con change impact end-to-end.

Il secondo è una **dipendenza da rappresentazioni method-ready**. Le tecniche di threat analysis più mature iniziano quando l'analista dispone già della vista richiesta: use case, goal model, problem frame, DFD, typed graph, asset model o test model. Gli studi sull'automated RE dimostrano che una parte di queste viste può essere generata da testo, ma proprio questi studi mostrano che l'output rimane candidato e che traceability, semantic correctness ed evolution sono problemi aperti.

Il terzo è una **eterogeneità dell'output**. Alcuni metodi producono threat scenario, altri anti-goal, altri candidate threat associati a componenti, altri test, altri security requirements o assurance argument. La letteratura non identifica un common finding boundary metodologicamente neutrale attraverso cui questi risultati possano essere

revisionati e poi trasformati uniformemente in requisiti di sicurezza mantenendo la provenance verso il metodo originario.

3.11 Research gap

Il research gap di questa tesi non è l'assenza di tecniche per identificare minacce. La letteratura contiene numerosi metodi e tool, inclusi approcci applicabili prima dell'implementazione. Non è neppure l'assenza di automazione: esistono trasformazioni model-to-threat, document-to-model, model-to-test e trace-link recovery. Infine, il gap non può essere formulato semplicemente come “manca una versione automatica di STRIDE”, perché STRIDE rappresenta solo uno dei possibili viewpoint e gli stessi studi mostrano che classificazioni e rappresentazioni differenti richiedono conoscenza differente.

Il gap emerge invece dall'intersezione di quattro discontinuità.

3.11.1 G1 - Dalla documentazione a un modello analizzabile governato

Gli approcci threat-oriented assumono generalmente un modello già pronto; gli approcci automated-RE possono generare artefatti da testo ma non stabiliscono una trasformazione completa, semanticamente affidabile, continuamente sincronizzata e methodology-neutral. Rimane quindi aperto il problema di costruire una rappresentazione canonica a partire dalla normale documentazione di progetto conservando, per ogni elemento derivato, fonte, versione, natura estratta o inferita e disposition di revisione.

In forma compatta:

raw/governed documentation $\neq$ trusted neutral analysis model.

La letteratura fornisce evidenza che la trasformazione sia possibile per artefatti specifici, non che essa sia già risolta per un core security-analyzable generale.

3.11.2 G2 - Un core neutrale per più metodologie

Le metodologie esaminate privilegiano scenario, goal, domain assumptions, componenti, flow, asset, failure mode o security property. Il problema non è trovare un super-modello che incorpori indiscriminatamente tutte le tassonomie. Una simile unione renderebbe il core dipendente dai metodi già conosciuti e ne renderebbe difficile l'estensione.

La letteratura del corpus non dimostra un boundary in cui:

- a) il progetto mantiene una sola conoscenza canonica del sistema;

- b) ciascun metodo dichiara gli input di cui ha bisogno;
- c) la semantica privata del metodo rimane nel plugin/overlay;
- d) l'esecuzione del metodo non modifica silenziosamente il modello canonico;
- e) metodi differenti possono essere applicati alla stessa baseline e confrontati attraverso un envelope comune.

Questa discontinuità corrisponde alla RQ2 ed è il contributo che la tesi intende verificare con due implementazioni concrete, STRIDE e STRIDE-AI. La scelta di due metodi non autorizzerà una generalizzazione a tutte le metodologie esistenti; permetterà di verificare se il boundary è operativo per due overlay distinti, uno vicino agli approcci software/component-centered e uno con framing asset-centered e AI/ML-specifico.

3.11.3 G3 - Dai risultati eterogenei a requisiti di sicurezza uniformi

Gli approcci requirements-oriented mostrano che threat e security requirement possono essere collegati, ma lo fanno all'interno della propria semantica. Gli automated threat-modeling tool producono liste di candidate con scope e granularità differenti. Gli approcci goal-oriented producono anti-goal e vulnerabilità; STRIDE-AI introduce asset, failure mode, proprietà e categorie di minaccia; gli executable misuse case producono trace eseguibili.

Manca nel corpus un boundary comune in cui il risultato specifico del metodo possa essere conservato nella sua forma nativa, trasformato in un finding metodologicamente neutrale per la revisione e, soltanto dopo acceptance, giustificare un requisito di sicurezza appartenente alla documentazione del prodotto. Questo boundary deve preservare la relazione inversa: dal requisito deve essere possibile risalire al finding, all'Analysis Record method-specific, agli elementi della Base Analysis e alle fonti documentali che hanno giustificato il percorso.

La discontinuità è quindi:

method-specific result $\rightarrow$ *reviewed common finding* $\rightarrow$ *governed security requirement*.

La letteratura supporta i singoli concetti, ma non questa composizione multi-metodo end-to-end.

3.11.4 G4 - Change-aware re-analysis

Traceability e regression literature mostrano il valore delle relazioni e della ripetizione dell'analisi, ma non forniscono una semantica unificata per propagare il cambiamento lungo tutta la pipeline DDTA. Un file modificato non rende necessariamente stale

ogni finding; una modifica a una tassonomia metodologica può invalidare un Analysis Record senza cambiare la Base Analysis; una correzione di provenance può modificare l'evidenza senza cambiare la semantica del sistema.

Il gap non è quindi il semplice versionamento degli artefatti. È la capacità di rappresentare dipendenze sufficientemente tipizzate da determinare quali elementi richiedano rivalutazione e quali possano rimanere validi. La catena candidata della tesi è:

source → *candidate* → *Base Analysis* → *analysis* → *finding* → *Security Requirement*.

La RQ4 valuterà questa proprietà attraverso modifiche controllate, non attraverso una pretesa di previsione universale di ogni impatto possibile.

3.12 Posizionamento di DDTA rispetto allo stato dell'arte

Le quattro discontinuità consentono di formulare in modo preciso il contributo candidato senza sovrapporlo alle metodologie esistenti. DDTA non propone una nuova tassonomia di minacce e non sostituisce STRIDE, STRIDE-AI, misuse case, goal-oriented analysis, PASTA o altri metodi. Propone un processo e un core che si collocano *prima, fra e dopo* questi metodi:

governed documentation → **reviewed methodology-neutral Base Analysis** → **isolated methodology overlay** → **method-specific Analysis Record** → **reviewed Common Finding** → **governed Security Requirement** → **change-aware re-analysis**

Ogni tratto della catena è motivato da una debolezza osservata nelle fonti. La candidate layer è motivata dagli errori delle trasformazioni automatiche e LLM; la neutralità della Base Analysis è motivata dall'eterogeneità dei viewpoint; il plugin boundary è motivato dalla natura method-specific di cataloghi, failure mode e regole; il Common Finding evita di incorporare la tassonomia del metodo nel requisito; provenance e review derivano dai problemi di traceability e grounding; lo stale state deriva dalla necessità di mantenere valida l'analisi quando cambiano fonti e strumenti.

Questa formulazione evita due claim che la letteratura non permette ancora di sostenere. Primo, non si afferma che la documentazione governata sia sufficiente a derivare sempre una Base Analysis completa: questa è materia della RQ1. Secondo, non si afferma che un core neutrale supporti qualunque metodologia: la RQ2 verrà valutata attraverso STRIDE e STRIDE-AI e la generalizzabilità oltre questi due casi resterà un limite esplicito.

3.13 Sintesi e collegamento al metodo di ricerca

Lo stato dell'arte permette di fissare sei conclusioni che costituiscono il punto di partenza sperimentale della tesi.

1. **Early threat analysis è consolidata, ma richiede rappresentazioni interpretate.** Le tecniche requirements-, scenario-, goal-, problem- e model-oriented dimostrano che il codice non è un prerequisito generale (Tuma et al., 2018), ma non trasformano automaticamente la normale documentazione in tali rappresentazioni.
2. **L'automazione esistente è boundary-specific.** Automated threat modeling opera dopo un typed model; model-based security testing opera dopo un security-enriched test model; automated RE opera prima, generando candidate artifact che richiedono verifica (Granata and Rak, 2024; Lonetti et al., 2023; Umar and Lano, 2024).
3. **La rappresentazione richiesta varia con il metodo.** Scenario, goal, component/flow e asset-centered approaches rendono visibili informazioni differenti. STRIDE-AI mostra concretamente che funzioni, failure mode e asset del lifecycle ML possono essere necessari oltre ai tradizionali componenti e data flow (Mauri and Damiani, 2022).
4. **Output plausibile non equivale ad artefatto accettato.** Gli studi LLM separano syntax, semantics, completeness, coverage, grounding e reproducibility e mostrano che una dimensione favorevole non garantisce le altre (Ferrari et al., 2024; Garaccione et al., 2025; Abualhaija et al., 2025; Angermeir et al., 2026).
5. **Traceability deve avere semantica e stato.** Link, provenance, versioning e responsabilità restano problemi aperti e le relazioni recuperate automaticamente devono essere distinguibili dalle relazioni revisionate (Mucha et al., 2024; Ruiz et al., 2023; Großer et al., 2022; Wang et al., 2024).
6. **Il gap è una composizione governata, non un nuovo metodo di threat identification.** Manca nel corpus un processo che colleghi documentazione governata, modello neutrale revisionato, più overlay metodologici, finding comuni, requisiti di sicurezza e re-analysis basata su provenance mantenendo separata la semantica dei diversi metodi.

Il Capitolo 4 tradurrà questo gap in un disegno di ricerca verificabile: casi con input documentale esplicito, reference model e risultati attesi dove sia possibile costruire un oracle difendibile; misure di agreement e missing/unexpected elements per la Base Analysis; verifica dell'invarianza del core durante l'esecuzione dei due plugin; confronto fra expected, missing e unexpected findings quando il caso lo consente; verifica della completezza della provenance fino ai Security Requirements; mutazioni controllate per

testare lo stale state. Costo, ROI e confronto economico restano fuori dalla valutazione primaria.

Il Capitolo 5 definirà poi la soluzione DDTA come proposta progettuale. Fino a quel punto, il research gap rimane una conclusione della sintesi bibliografica, mentre l'efficacia della soluzione proposta rimane un'ipotesi da valutare.

Bibliografia

- O. C. Z. Gotel and A. C. W. Finkelstein. An Analysis of the Requirements Traceability Problem. In *Proceedings of the First International Conference on Requirements Engineering*, pages 94–101, 1994. doi:10.1109/ICRE.1994.292398.
- G. Sindre and A. L. Opdahl. Eliciting Security Requirements with Misuse Cases. *Requirements Engineering*, 10(1):34–44, 2005. doi:10.1007/s00766-004-0194-4.
- A. van Lamsweerde. Elaborating Security Requirements by Construction of Intentional Anti-Models. In *Proceedings of the 26th International Conference on Software Engineering*, pages 148–157, 2004. doi:10.1109/ICSE.2004.1317437.
- T. Lodderstedt, D. Basin, and J. Doser. SecureUML: A UML-Based Modeling Language for Model-Driven Security. In *UML 2002*, pages 426–441, 2002. doi:10.1007/3-540-45800-X_33.
- D. Basin, J. Doser, and T. Lodderstedt. Model Driven Security: From UML Models to Access Control Infrastructures. *ACM Transactions on Software Engineering and Methodology*, 15(1):39–91, 2006. doi:10.1145/1125808.1125810.
- J. Mucha, A. Kaufmann, and D. Riehle. A systematic literature review of pre-requirements specification traceability. *Requirements Engineering*, 29(2):119–141, 2024. doi:10.1007/s00766-023-00412-z.
- M. Ruiz, J. Y. Hu, and F. Dalpiaz. Why don't we trace? A study on the barriers to software traceability in practice. *Requirements Engineering*, 28(4):619–637, 2023. doi:10.1007/s00766-023-00408-9.
- K. Großer, V. Riediger, and J. Jurjens. Requirements document relations: A reuse perspective on traceability through standards. *Software and Systems Modeling*, 21:2133–2169, 2022. doi:10.1007/s10270-021-00958-y.
- B. Wang, Z. Zou, H. Wan, Y. Li, Y. Deng, and X. Li. An empirical study on the state-of-the-art methods for requirement-to-code traceability link recovery. *Journal of King Saud University - Computer and Information Sciences*, 36(6):102118, 2024. doi:10.1016/j.jksuci.2024.102118.

- T. Hey, J. Keim, and S. Corallo. Requirements Classification for Traceability Link Recovery. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, pages 155–167, 2024. doi:10.1109/RE59067.2024.00024.
- K. Tuma, G. Calikli, and R. Scandariato. Threat analysis of software systems: A systematic literature review. *Journal of Systems and Software*, 144:275–294, 2018. doi:10.1016/j.jss.2018.06.073.
- C. B. Haley, R. Laney, J. D. Moffett, and B. Nuseibeh. Security Requirements Engineering: A Framework for Representation and Analysis. *IEEE Transactions on Software Engineering*, 34(1):133–153, 2008. doi:10.1109/TSE.2007.70754.
- J. Whittle, D. Wijesekera, and M. Hartong. Executable Misuse Cases for Modeling Security Concerns. In *Proceedings of the 30th International Conference on Software Engineering*, pages 121–130, 2008. doi:10.1145/1368088.1368106.
- D. Hatebur, M. Heisel, and H. Schmidt. Security Engineering Using Problem Frames. In *Emerging Trends in Information and Communication Security*, LNCS 3995, pages 238–253. Springer, 2006. doi:10.1007/11766155_17.
- D. Granata and M. Rak. Systematic analysis of automated threat modelling techniques: Comparison of open-source tools. *Software Quality Journal*, 32:125–161, 2024. doi:10.1007/s11219-023-09634-4.
- F. Lonetti, A. Bertolino, and F. Di Giandomenico. Model-based security testing in IoT systems: A Rapid Review. *Information and Software Technology*, 164:107326, 2023. doi:10.1016/j.infsof.2023.107326.
- M. A. Umar and K. Lano. Advances in automated support for requirements engineering: a systematic literature review. *Requirements Engineering*, 29:177–207, 2024. doi:10.1007/s00766-023-00411-0.
- E. Souza, A. Moreira, and M. Goulão. Deriving architectural models from requirements specifications: A systematic mapping study. *Information and Software Technology*, 109:26–39, 2019. doi:10.1016/j.infsof.2019.01.004.
- A. Ferrari, S. Abualhaija, and C. Arora. Model Generation with LLMs: From Requirements to UML Sequence Diagrams. In *2024 IEEE 32nd International Requirements Engineering Conference Workshops*, pages 291–300, 2024. doi:10.1109/REW61692.2024.00044.
- G. Garaccione, D. M. Calabrese, R. Coppola, and L. Ardito. A comparison of different Large Language Models for the generation of UML class diagrams. In *2025 ACM/IEEE 28th MODELS Companion*, pages 541–545, 2025. doi:10.1109/MODELS-C68889.2025.00078.

- S. Abualhaija, M. Ceci, N. Sannier, D. Bianculli, S. Lannier, M. Siclari, O. Voordeckers, and S. Tosza. LLM-assisted Extraction of Regulatory Requirements: A Case Study on the GDPR. In *2025 IEEE 33rd International Requirements Engineering Conference*, pages 142–154, 2025. doi:10.1109/RE63999.2025.00023.
- F. Angermeir, M. Amougou, M. Kreitz, A. Bauer, M. Linhuber, D. Fucci, F. Moyón C., D. Mendez, and T. Gorschek. Reflections on the Reproducibility of Commercial LLM Performance in Empirical Software Engineering Studies. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. doi:10.1145/3744916.3773207.
- L. Mauri and E. Damiani. Modeling Threats to AI-ML Systems Using STRIDE. *Sensors*, 22(17):6662, 2022. doi:10.3390/s22176662.